Greedy and Divide-and-Conquer Algorithms for Real-World Problems

Ilan Danial
Dept. of Computer & Information Science & Engineering
University of Florida
Gainesville, FL, USA
ilan.danial@ufl.edu

[Partner: name]
Dept. of Computer & Information Science & Engineering
University of Florida
Gainesville, FL, USA
[Partner: email]

***Abstract*—[Partner: one or two sentences on the greedy problem and result.] Map-search sites cap the number of listings shown per window, so collecting every listing in a city means choosing windows that never overflow. We model this as a point set on an integer grid and give a divide-and-conquer algorithm that recursively splits overflowing windows into quadrants. We prove it correct and bound its recursion depth by $\lceil \log_2 W \rceil$ for a map of W grid units; every statement is machine-checked in Lean 4. For evenly spread listings it makes $\Theta(n/k)$ requests and $\Theta(n \log(n/k))$ work, which experiments confirm on synthetic data and on 261 real Airbnb listings.**

***Index Terms*—[Partner: greedy keywords], divide and conquer, quadtree, web scraping, formal verification**

I. Introduction

This report applies two algorithm-design techniques to problems that arose outside the classroom. For each we state the problem, abstract it, give an algorithm with a running-time analysis and a proof of correctness, explain the algorithm in the problem's own terms, and check the analysis experimentally.

[Partner: one paragraph introducing the greedy problem and summarizing the result.]

The second problem comes from web data collection. A map-search service shows at most k listings for any map window, so a scraper that needs every listing in a city must split the map into windows that never overflow, using as few requests as it can. Section III models the map as an integer grid and the service as a capped oracle, solves the problem with a divide-and-conquer recursion on quadrants, proves the recursion correct and bounds its depth by $\lceil \log_2 W \rceil$, and verifies the predicted $\Theta(n \log(n/k))$ cost experimentally. Every statement in the correctness section is also machine-checked in Lean 4 (Appendix B).

II. Greedy: [Partner: section title]

[Partner: the greedy half: problem, abstraction, algorithm, running time, proof of correctness, explanation in the problem domain, experiments. Code goes in an appendix after Appendix B; LLM prompts and results in an appendix after Appendix C.]

III. Divide and Conquer: Capped Map Search

A. Problem

Airbnb's map search returns the listings inside the visible map window, but never more than a fixed number of them. When a window holds more, the rest are cut off and cannot be reached from that window. A market study of a city needs *every* listing, so a scraper must choose map windows small enough that none is truncated, using as few searches as possible. This problem came up while scraping Miami, FL and Rincón, PR [1]; a single city-wide window returned only a small fraction of the listings.

B. Abstraction

Reported coordinates have fixed precision (Airbnb gives at most five decimals), so we measure locations in grid units and model the map as $\mathbb{Z}^2$. Let P be a set of n *listings*, each with a location in $\mathbb{Z}^2$. Several listings may share a location; let m be the largest number sharing one. A *window* is a half-open rectangle $r = [x_0, x_1) \times [y_0, y_1)$ with integer corners; its *side* is $\max(x_1 - x_0,\ y_1 - y_0)$, and P_r denotes the listings located in r. The search region R is a window with $P_R = P$ and side W.

The search service is an oracle with cap $k \geq 1$:

$$Q_k(r) = \begin{cases} P_r & \text{if } |P_r| \leq k, \\ \textsc{overflow} & \text{otherwise.} \end{cases} \tag{1}$$

The problem is to compute P from R and Q_k using as few oracle calls (web requests) as possible.

Splitting a window at $x_m = \lfloor (x_0 + x_1)/2 \rfloor$ and $y_m = \lfloor (y_0 + y_1)/2 \rfloor$ gives four *quadrants* $r_1, \ldots, r_4$. Splitting recursively yields a *quadtree* [2], [3] whose leaves are the windows actually read.

C. Algorithm

Collect (Fig. 1) queries a window. If it is not truncated, its listings are the answer. Otherwise the window is divided into four quadrants, each is solved recursively, and the answers are united. The full answer is $\textsc{Collect}(R)$.

1: **function** COLLECT(r)
2: $A \leftarrow Q_k(r)$
3: **if** $A \neq$ OVERFLOW **then**
4: **return** A {window fits under the cap}
5: **end if**
6: $r_1, \ldots, r_4 \leftarrow$ quadrants of r {divide}
7: **return** $\bigcup_{i=1}^{4}$ COLLECT(r_i) {conquer, combine}

Fig. 1. Enumerating the listings in window r through a capped oracle Q_k.

D. Correctness

We first show that any answer COLLECT produces is right (Theorem 1), then that it always produces one when $m \le k$ (Theorem 2).

Lemma 1*: Every location in r lies in exactly one quadrant of r, and no location outside r lies in any quadrant.*

Proof: Since $x_0 \le x_1$ we have $x_0 \le x_m \le x_1$, and likewise for y_m. For $(x, y) \in r$, exactly one of $x < x_m$ and $x \ge x_m$ holds, and exactly one of $y < y_m$ and $y \ge y_m$; the pair selects exactly one quadrant. A location outside r violates one of the four bounds of r, and every quadrant keeps or tightens each of them. ∎

Because the windows are half-open, a listing on a split line falls in exactly one quadrant, so it is neither lost nor counted twice.

Theorem 1 (partial correctness)*: If* COLLECT(r) *terminates, it returns P_r, every leaf window holds at most k listings, and every location in r lies in exactly one leaf window (and no location outside r in any).*

Proof: Strong induction on the height h of the recursion tree. If $h = 0$, the oracle did not overflow, so $|P_r| \le k$ and line 4 returns $Q_k(r) = P_r$; the only leaf is r itself. If $h > 0$, the algorithm recurses on the quadrants $r_1, \ldots, r_4$, whose recursion trees are shorter than h, so by induction COLLECT(r_i) $= P_{r_i}$, the leaves under r_i hold at most k listings, and each location is in exactly $[p \in r_i]$ of them. By Lemma 1 each listing of P_r lies in exactly one r_i, so $\bigcup_i P_{r_i} = P_r$; and the number of leaves containing a location p is $\sum_i [p \in r_i] = [p \in r]$, again by Lemma 1. ∎

Lemma 2*: A window of side at most* 1 *contains at most one location. Hence if $m \le k$, such a window holds at most k listings and is never split.*

Proof: If $x_0 \le x < x_1 \le x_0 + 1$ then $x = x_0$, and likewise $y = y_0$, so P_r consists of listings at (x_0, y_0), of which there are at most $m \le k$. ∎

Lemma 3*: If r has side at most $2B$, each quadrant of r has side at most B.*

Proof: With $S = x_1 - x_0 \le 2B$, the two horizontal halves have widths $x_m - x_0 = \lfloor S/2 \rfloor$ and $x_1 - x_m = \lceil S/2 \rceil$, both at most B; the same holds vertically. ∎

Theorem 2 (termination and depth)*: If $m \le k$ and r has side at most 2^D, then* COLLECT(r) *terminates with recursion depth at most D. In particular* COLLECT(R) *has height at most* $\lceil \log_2 W \rceil$.

Proof: Induction on D. If $D = 0$, the side is at most 1, so by Lemma 2 the oracle answers and the recursion stops at depth 0. For $D + 1$: if the oracle answers, we are done; otherwise the quadrants have side at most 2^D by Lemma 3, so by induction each recursive call terminates within depth D, and COLLECT(r) within depth $D + 1$. ∎

Corollary 1 (total correctness)*: If $m \le k$,* COLLECT(R) *terminates within depth $\lceil \log_2 W \rceil$ and returns P; its leaf windows partition R and each holds at most k listings.*

Proof: Theorem 2 gives termination, Theorem 1 the rest. ∎

The condition $m \le k$ is also necessary. If more than k listings share a location, every window containing it overflows, and no strategy using Q_k can reveal them.

Every statement in this subsection is machine-checked in Lean 4 [4] (Appendix B), under the following correspondence:

Lemma 1	`quadrants_partition`
Theorem 1	`collect_correct`
Lemma 2	`unit_window_fits`
Lemma 3	`quadrants_side`
Theorem 2	`collect_terminates`
Corollary 1	`collect_total`

Lean accepts a recursive definition only with a termination argument, so its COLLECT takes the remaining depth budget as an explicit parameter and returns $\perp$ if the budget runs out. Theorem 2 says that the budget $\lceil \log_2 W \rceil$ never runs out. The closest existing formalizations, region quadtrees in Isabelle [5], [6] and k-d trees in Isabelle and Coq [7], [8], obtain termination from a fixed grid exponent or from the point count shrinking at each median split. Neither covers a tree that splits space over arbitrary points.

E. Running Time

Let N be the number of oracle calls and T the total work when a node with $|P_r|$ listings spends $\Theta(|P_r| + 1)$ time returning or distributing them. Let h be the tree height.

Windows at one depth are disjoint (Lemma 1), so each level holds at most n listings and at most $\lfloor n/(k+1) \rfloor$ split windows, each holding at least $k+1$ listings. Internal nodes sit at depths $0, \ldots, h-1$ and have four children each, so

$$\lceil n/k \rceil \le N \le 1 + 4h\Big\lfloor \frac{n}{k+1} \Big\rfloor, \qquad T = O\big(n(h+1)\big). \quad (2)$$

The lower bound holds because the leaves cover P with at most k listings each. By Theorem 2, $h \le \lceil \log_2 W \rceil$, so $T =$



Fig. 2. Scaling with n at $k = 50$, for uniform and clustered listings. (a) Running time with the least-squares fit $c\,n\log_2(n/k)$ ($R^2 = 0.995$ uniform, 0.996 clustered). (b) Running time divided by $n\log_2(n/k)$; a flat curve confirms the growth rate. (c) Requests issued, against the lower bound n/k.

$O(n \log W)$: the cost is governed by the coordinate resolution, not by n alone.

In the balanced case, where each split sends about $n/4$ listings to each quadrant (true in expectation for uniformly random locations),

$$T(n) = 4\,T(n/4) + \Theta(n) \quad (n > k), \tag{3}$$

with $T(n) = \Theta(1)$ for $n \le k$. With $a = b = 4$ and $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, case 2 of the Master Theorem [9] gives $T(n) = \Theta(n \log n)$. Unrolling the recursion shows the dependence on k: the tree is complete with $L = \lceil \log_4(n/k) \rceil$ levels below the root, each costing $\Theta(n)$, so

$$T(n) = \Theta\left(n \log \frac{n}{k}\right), \tag{4}$$

$$N = \sum_{i=0}^{L} 4^i = \frac{4^{L+1} - 1}{3} = \Theta\left(\frac{n}{k}\right), \tag{5}$$

within a constant factor of the n/k requests any strategy must make.

F. In the Language of the Problem

Search the whole city in one window. If the results fit on the pages the site shows, record them. Otherwise cut the window into four equal tiles and repeat on each. A listing exactly on a cut goes to the tile right of or above it, so it is recorded once. Dense neighborhoods are zoomed into repeatedly, while open water is dismissed after one search. By (5), the number of searches is proportional to n/k, the number of listings divided by the number shown per search, and the zoom never goes deeper than $\lceil \log_2 W \rceil$ levels. With five-decimal coordinates over a one-degree span, $W = 10^5$ and 17 levels suffice. The method fails only at an address with more listings than the cap, such as an apartment building whose units share rounded coordinates. No zoom level separates them, which is why Corollary 1 needs $m \le k$. In Rincón at most 7 listings share a location, so any $k \ge 7$ works.

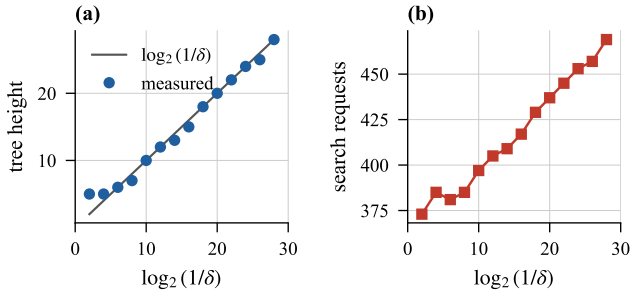


Fig. 3. Effect of packing $k+1$ listings into a square of side δ ($n = 10{,}000$, $k = 50$): (a) tree height, (b) requests.

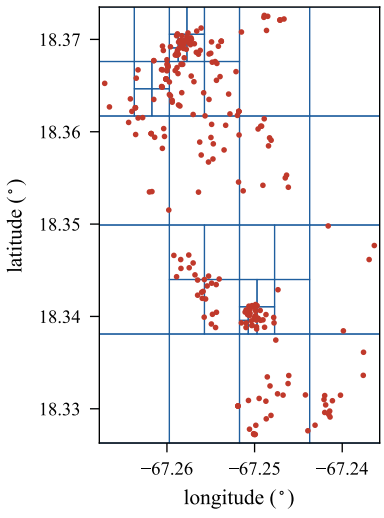


Fig. 4. Windows chosen for 261 listings in Rincón, PR ($k = 20$).

G. Experiments

Setup. COLLECT was implemented in Python 3.14 (Appendix A) and run on an AMD Ryzen 7 9800X3D. The oracle

TABLE I
REQUESTS ON UNIFORM INPUTS ($k = 50$) VS. THE PREDICTION (5)

n	measured	predicted	height
1,024	85	85	3
4,096	333	341	4
16,384	1,337	1,365	5
65,536	5,293	5,461	6
262,144	21,197	21,845	7
524,288	21,917	21,845	8

is simulated. Each node partitions the listings it holds among its quadrants, which is the $\Theta(|P_r|)$ step of (3). Coordinates are double-precision numbers in the unit square, i.e. the grid model at resolution 2^{-52}; the implementation also stops at windows narrower than 10^{-9} and flags them, which never happened. We used $k = 50$ and $n = 2^{10}, \ldots, 2^{19}$ listings, either uniform or drawn from 20 Gaussian clusters ($\sigma = 0.02$). Each size was run five times on fresh random inputs; we report the fastest run with garbage collection disabled, as `timeit` does [10]. Every run was checked to return each listing exactly once.

Running time. Fig. 2(a) shows measured time against the fitted model. Fig. 2(b) divides time by $n \log_2(n/k)$. Over a $512\times$ range of n the ratio stays between 26 and 43 ns (uniform) and between 36 and 48 ns (clustered), confirming $\Theta(n \log(n/k))$. The zig-zag arises because the tree gains a level only when n grows about fourfold, so the true cost rises in steps. The slight rise at the largest sizes is consistent with cache effects. Clustered inputs cost about 20% more because their trees are two levels taller.

Requests. Table I compares requests on uniform inputs with the exact balanced count (5); they agree within 3.1%. Fig. 2(c) shows the count growing linearly, parallel to the lower bound n/k.

Crowding. To test the depth bound, we fixed $n = 10{,}000$ uniform listings and added $k + 1 = 51$ listings inside a square of side $\delta = 2^{-2}, \ldots, 2^{-28}$. Isolating that square requires windows of side about δ, i.e. a grid of $W = 2^e$ units when $\delta = 2^{-e}$. Fig. 3 shows the tree height tracking $\log_2(1/\delta) = \log_2 W$ (Theorem 2) although n is fixed, and each extra level adding about four requests, the $4h$ term of (2).

Real data. Fig. 4 runs the algorithm with $k = 20$ on 261 Airbnb listings from Rincón, PR (scraped March 29, 2023 [1]). It used 57 requests, 43 windows and height 5, within the bounds $14 \le N \le 241$ of (2). The two dense clusters are split repeatedly; the outskirts are covered by a few large windows.

IV. CONCLUSION

[Partner: one paragraph concluding the greedy half.]

For the capped map search, splitting overflowing windows into quadrants collects every listing with a number of requests within a constant factor of the n/k lower bound when listings are spread evenly, and never recurses deeper than $\lceil \log_2 W \rceil$ levels on a map of W grid units. The one assumption, that no single location holds more than k listings, is necessary for any strategy that only sees capped search results. Measured running times match the $\Theta(n \log(n/k))$ analysis within a few percent over three orders of magnitude, and the correctness argument is machine-checked.

REFERENCES

[1] I. Danial, "airbnb_data: Airbnb listing scraper," https://github.com/IlanDanial/airbnb_data, 2023, accessed Oct. 2026.

[2] R. A. Finkel and J. L. Bentley, "Quad trees: A data structure for retrieval on composite keys," *Acta Informatica*, vol. 4, no. 1, pp. 1–9, 1974.

[3] H. Samet, "The quadtree and related hierarchical data structures," *ACM Computing Surveys*, vol. 16, no. 2, pp. 187–260, 1984.

[4] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, ser. LNCS, vol. 12699. Springer, 2021, pp. 625–635.

[5] T. Nipkow, "Region quadtrees verified," in *Logics and Type Systems in Theory and Practice*, ser. LNCS. Springer, 2024.

[6] ——, "Region quadtrees," Archive of Formal Proofs, https://isa-afp.org/entries/Region_Quadtrees.html, 2024.

[7] M. Rau, "Multidimensional binary search trees," Archive of Formal Proofs, https://isa-afp.org/entries/KD_Tree.html, 2019.

[8] N. A. Hamid, "(nearest) neighbors you can rely on: Formally verified k-d tree construction and search in Coq," arXiv:2311.10965, 2023.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[10] Python Software Foundation, "timeit — measure execution time of small code snippets," https://docs.python.org/3/library/timeit.html, accessed Oct. 2026.

APPENDIX A
DIVIDE-AND-CONQUER CODE

The algorithm (`quadtree.py`) and the measurements behind Figs. 2 and 3 and Table I (`experiments.py`). The plotting script, the unit tests, the Rincón data file and the raw measurements are in the project repository.[1]

Listing 1. quadtree.py

```
"""Adaptive quadtree decomposition for capped map search (divide and conquer).

A map-search service (e.g. Airbnb's map view) returns the listings inside a
rectangular window only when the window holds at most k of them; otherwise it
truncates the results. To enumerate every listing in a region we split the
window into four quadrants and recurse until every window fits under the cap.

Abstraction: P is a set of n points in a rectangle R. The leaves of the
recursion form a partition of R into half-open rectangles, each holding at
most k points of P, and the union of their points is exactly P.
"""

import math
from dataclasses import dataclass, field


@dataclass(frozen=True)
class Rect:
    """Half-open rectangle [x0, x1) x [y0, y1).

    Half-open bounds guarantee that a point on a split line belongs to exactly
    one quadrant, so no listing is reported twice or skipped.
    """

    x0: float
    y0: float
    x1: float
    y1: float

    def contains(self, p):
        return self.x0 <= p[0] < self.x1 and self.y0 <= p[1] < self.y1

    def area(self):
        return (self.x1 - self.x0) * (self.y1 - self.y0)

    def quadrants(self, xm, ym):
        # Order matches the bucket index (x >= xm) + 2 * (y >= ym) in _collect.
        return (
            Rect(self.x0, self.y0, xm, ym),  # 0: lower-left
            Rect(xm, self.y0, self.x1, ym),  # 1: lower-right
            Rect(self.x0, ym, xm, self.y1),  # 2: upper-left
            Rect(xm, ym, self.x1, self.y1),  # 3: upper-right
        )


@dataclass
class Result:
    points: list = field(default_factory=list)  # every listing found
    leaves: list = field(default_factory=list)  # windows that fit under the cap
    queries: int = 0  # search requests issued (one per recursion node)
    max_depth: int = 0  # height of the quadtree
    unresolved: int = 0  # windows that hit min_width while still over the cap


def bounding_rect(points):
    """Smallest half-open rectangle containing every point.

    The upper bounds are nudged to the next float so the points with the
    largest x or y still fall inside the half-open box.
    """
    xs = [p[0] for p in points]
    ys = [p[1] for p in points]
    return Rect(min(xs), min(ys),
                math.nextafter(max(xs), math.inf), math.nextafter(max(ys), math.inf))


def collect(rect, points, k, min_width=1e-9):
    """Return every point in `rect` using windows of at most k points.

    `points` is the ground truth the simulated search service holds for
    `rect`; the algorithm only learns it through the capped query on each
    window. If more than k points share (nearly) the same location no window
```

[1] https://github.com/IlanDanial/P1Alg

```
    can isolate them, so recursion stops once a window is narrower than
    `min_width` and the window is counted as unresolved.
    """
    if k < 1:
        raise ValueError("k must be at least 1")
    result = Result()
    _collect(rect, points, k, min_width, 0, result)
    return result


def _collect(rect, points, k, min_width, depth, result):
    result.queries += 1
    result.max_depth = max(result.max_depth, depth)

    # Base case: the service shows the whole window.
    if len(points) <= k:
        result.points.extend(points)
        result.leaves.append(rect)
        return

    # Cannot zoom any further: keep the truncated page the service shows.
    if rect.x1 - rect.x0 <= min_width or rect.y1 - rect.y0 <= min_width:
        result.points.extend(points[:k])
        result.leaves.append(rect)
        result.unresolved += 1
        return

    # Divide: split the window at its midpoint into four quadrants.
    xm = (rect.x0 + rect.x1) / 2
    ym = (rect.y0 + rect.y1) / 2
    buckets = ([], [], [], [])
    for p in points:
        buckets[(p[0] >= xm) + 2 * (p[1] >= ym)].append(p)

    # Conquer each quadrant; combining is just appending to the shared result.
    for quad, bucket in zip(rect.quadrants(xm, ym), buckets):
        _collect(quad, bucket, k, min_width, depth + 1, result)
```

Listing 2. experiments.py

```
"""Running-time experiments for quadtree.collect.

Three experiments, each written to results/*.csv:
  scaling_uniform.csv, scaling_clustered.csv  time, requests and tree height
                                              as n grows (balanced case)
  depth_vs_delta.csv                          tree height as k+1 listings are
                                              packed ever closer together
plots.py turns the CSV files into the figures in the report.

Run: python experiments.py   (about a minute)
"""

import csv
import gc
import math
import random
import statistics
import time
from collections import Counter
from pathlib import Path

from quadtree import Rect, collect

RESULTS = Path(__file__).parent / "results"
K = 50  # listings a search shows before truncating (a few result pages)
SIZES = [2**e for e in range(10, 20)]  # 1,024 .. 524,288 listings
TRIALS = 5  # random inputs per size
UNIT = Rect(0.0, 0.0, 1.0, 1.0)


# -- inputs -------------------------------------------------------------

def uniform(n, rng):
    """n listings spread evenly over the unit square."""
    return [(rng.random(), rng.random()) for _ in range(n)]


def clustered(n, rng, centers=20, spread=0.02):
    """n listings concentrated around a few neighborhoods (Gaussian clusters)."""
    cs = [(rng.random(), rng.random()) for _ in range(centers)]
    pts = []
    while len(pts) < n:
        cx, cy = rng.choice(cs)
```

```
        x, y = rng.gauss(cx, spread), rng.gauss(cy, spread)
        if 0.0 <= x < 1.0 and 0.0 <= y < 1.0:
            pts.append((x, y))
    return pts


# -- predictions from the analysis -------------------------------------------

def model(n):
    """Predicted work n log2(n/k), from T(n) = 4T(n/4) + Theta(n)."""
    return n * math.log2(n / K)


def balanced_queries(n):
    """Requests when every split is even: a complete 4-ary tree of L+1 levels."""
    levels = 0
    while K * 4**levels < n:
        levels += 1
    return (4 ** (levels + 1) - 1) // 3


def fit_constant(rows):
    """Least-squares c for time = c * model(n) over the rows, and the fit's R^2."""
    f = [model(r["n"]) for r in rows]
    t = [r["time_s"] for r in rows]
    c = sum(a * b for a, b in zip(f, t)) / sum(a * a for a in f)
    mean = statistics.mean(t)
    ss_res = sum((ti - c * fi) ** 2 for fi, ti in zip(f, t))
    ss_tot = sum((ti - mean) ** 2 for ti in t)
    return c, 1 - ss_res / ss_tot


# -- measurement -------------------------------------------------------------

def time_collect(pts):
    """Time collect() on pts, then check it returned every listing exactly once."""
    # Like timeit, keep the garbage collector from adding pauses to the timing.
    gc.collect()
    gc.disable()
    try:
        start = time.perf_counter()
        result = collect(UNIT, pts, K)
        elapsed = time.perf_counter() - start
    finally:
        gc.enable()
    assert result.unresolved == 0 and Counter(result.points) == Counter(pts)
    return elapsed, result


def run_scaling(name, gen):
    """Time collect() for each n in SIZES on TRIALS fresh inputs from gen.

    Reports the fastest time (the least-disturbed run, as timeit recommends)
    and the median request count and tree height.
    """
    rows = []
    for n in SIZES:
        times, queries, depths = [], [], []
        for t in range(TRIALS):
            pts = gen(n, random.Random(1000 * n + t))
            elapsed, result = time_collect(pts)
            times.append(elapsed)
            queries.append(result.queries)
            depths.append(result.max_depth)
        row = {
            "n": n,
            "time_s": min(times),
            "queries": int(statistics.median(queries)),
            "balanced_queries": balanced_queries(n),
            "max_depth": int(statistics.median(depths)),
        }
        rows.append(row)
        print(f"{name:9s} n={n:7d}  time={row['time_s']:.4f}s  "
              f"queries={row['queries']}  depth={row['max_depth']}")
    write_csv(RESULTS / f"scaling_{name}.csv", rows)
    return rows


def run_depth(n=10_000, seed=7):
    """Pack k+1 listings (a 'building') into a square of side delta = 2^-e.

    n stays fixed, so any growth in height and requests comes from the
    crowding alone. The building is placed away from the map's edges so the
```

```
    quadrants that isolate it are not clipped by the border.
    """
    rows = []
    for e in range(2, 29, 2):  # stays above collect's min_width of 1e-9 ~ 2^-30
        delta = 2.0**-e
        rng = random.Random(seed)
        pts = uniform(n - (K + 1), rng)
        cx, cy = 0.3 + 0.4 * rng.random(), 0.3 + 0.4 * rng.random()
        pts += [(cx + delta * rng.random(), cy + delta * rng.random()) for _ in range(K + 1)]
        elapsed, result = time_collect(pts)
        rows.append({"log2_inv_delta": e, "time_s": elapsed,
                     "queries": result.queries, "max_depth": result.max_depth})
        print(f"depth     delta=2^-{e:<2d} queries={result.queries}  depth={result.max_depth}")
    write_csv(RESULTS / "depth_vs_delta.csv", rows)
    return rows


def write_csv(path, rows):
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)


if __name__ == "__main__":
    RESULTS.mkdir(exist_ok=True)
    run_scaling("uniform", uniform)
    run_scaling("clustered", clustered)
    run_depth()
```

APPENDIX B
MACHINE-CHECKED PROOF

Section III-D in Lean 4.35, using only the core library: Lemma 1 (`quadrants_partition`), Theorem 1 (`collect_correct`), Lemma 2 (`unit_window_fits`), Lemma 3 (`quadrants_side`), Theorem 2 (`collect_terminates`) and Corollary 1 (`collect_total`). `lake build` in `divide_and_conquer/proof` checks it; the theorems depend on no axioms beyond Lean's standard three (`propext`, `Classical.choice`, `Quot.sound`).

Listing 3. proof/Proof/Collect.lean

```
/-!
# Capped map search: machine-checked correctness of `collect`

Formalizes Section II-D of the report, statement for statement. Coordinates
are integers: reported coordinates have fixed precision, so the plane is a
grid. Only Lean's core library is used; every arithmetic step is discharged by
`omega`.

| Report      | Lean                    |
|-------------|-------------------------|
| Lemma 1     | `quadrants_partition`   |
| Theorem 1   | `collect_correct`       |
| Lemma 2     | `unit_window_fits`      |
| Lemma 3     | `quadrants_side`        |
| Theorem 2   | `collect_terminates`    |
| Corollary 1 | `collect_total`         |

`collect` carries its remaining depth budget as an explicit argument and
returns `none` if the budget runs out. Theorem 2 shows that with budget
`⌈log₂ W⌉` this never happens when no location holds more than `k` listings.
-/

namespace Quadtree

/-- A half-open window `[x0, x1) × [y0, y1)`. -/
structure Window where
  x0 : Int
  x1 : Int
  y0 : Int
  y1 : Int

/-- The location `p` lies in window `w`. -/
def inWin (p : Int × Int) (w : Window) : Prop :=
  w.x0 ≤ p.1 ∧ p.1 < w.x1 ∧ w.y0 ≤ p.2 ∧ p.2 < w.y1

instance (p : Int × Int) (w : Window) : Decidable (inWin p w) := by
  unfold inWin; infer_instance

/-- The four quadrants of `w`, split at `⌊(x0+x1)/2⌋` and `⌊(y0+y1)/2⌋`. -/
```

```
def q1 (w : Window) : Window := ⟨w.x0, (w.x0 + w.x1) / 2, w.y0, (w.y0 + w.y1) / 2⟩
def q2 (w : Window) : Window := ⟨(w.x0 + w.x1) / 2, w.x1, w.y0, (w.y0 + w.y1) / 2⟩
def q3 (w : Window) : Window := ⟨w.x0, (w.x0 + w.x1) / 2, (w.y0 + w.y1) / 2, w.y1⟩
def q4 (w : Window) : Window := ⟨(w.x0 + w.x1) / 2, w.x1, (w.y0 + w.y1) / 2, w.y1⟩

/-- `1` if `p` lies in `w`, else `0`. -/
def ind (p : Int × Int) (w : Window) : Nat := if inWin p w then 1 else 0

/-! ## Lemma 1: the quadrants partition the window -/

/-- **Lemma 1.** A location in `w` lies in exactly one quadrant; a location
outside `w` lies in none. -/
theorem quadrants_partition (p : Int × Int) (w : Window) :
    ind p (q1 w) + ind p (q2 w) + ind p (q3 w) + ind p (q4 w) = ind p w := by
  -- Decide each of the five memberships, then the remaining goal is linear arithmetic.
  by_cases h0 : inWin p w <;> by_cases h1 : inWin p (q1 w) <;> by_cases h2 : inWin p (q2 w) <;>
    by_cases h3 : inWin p (q3 w) <;> by_cases h4 : inWin p (q4 w) <;>
    simp only [ind, h0, h1, h2, h3, h4, ite_true, ite_false] <;>
    simp only [inWin, q1, q2, q3, q4] at h0 h1 h2 h3 h4 <;> omega

/-- Lemma 1 in its set form: `p` is in `w` iff it is in some quadrant. -/
theorem quadrants_cover (p : Int × Int) (w : Window) :
    inWin p w ↔ inWin p (q1 w) ∨ inWin p (q2 w) ∨ inWin p (q3 w) ∨ inWin p (q4 w) := by
  simp only [inWin, q1, q2, q3, q4]
  omega

/-! ## The algorithm -/

variable {α : Type} (P : List α) (loc : α → Int × Int) (k : Nat)

/-- `P_w`: the listings located in `w`. -/
def listingsIn (w : Window) : List α :=
  P.filter (fun a => decide (inWin (loc a) w))

/-- The capped search: the listings in `w` if there are at most `k`, else overflow. -/
def oracle (w : Window) : Option (List α) :=
  if (listingsIn P loc w).length ≤ k then some (listingsIn P loc w) else none

/-- `Collect` from the report, with an explicit depth budget. Returns the
listings found and the leaf windows, or `none` if the budget runs out. -/
def collect : Nat → Window → Option (List α × List Window)
  | 0, w =>
    match oracle P loc k w with
    | some A => some (A, [w])
    | none => none
  | fuel + 1, w =>
    match oracle P loc k w with
    | some A => some (A, [w])
    | none =>
      match collect fuel (q1 w), collect fuel (q2 w),
            collect fuel (q3 w), collect fuel (q4 w) with
      | some (o1, l1), some (o2, l2), some (o3, l3), some (o4, l4) =>
        some (o1 ++ o2 ++ o3 ++ o4, l1 ++ l2 ++ l3 ++ l4)
      | _, _, _, _ => none

/-- Number of leaves in `leaves` containing `p`. -/
def leafCount (p : Int × Int) (leaves : List Window) : Nat :=
  leaves.countP (fun l => decide (inWin p l))

/-- The three properties Theorem 1 asserts of an answer `(out, leaves)` for `w`:
`out` is exactly `P_w`; every leaf is under the cap; every location in `w` lies
in exactly one leaf and every location outside `w` in none. -/
def Correct (w : Window) (out : List α) (leaves : List Window) : Prop :=
  (∀ a, a ∈ out ↔ a ∈ P ∧ inWin (loc a) w) ∧
  (∀ l ∈ leaves, (listingsIn P loc l).length ≤ k) ∧
  (∀ p, leafCount p leaves = ind p w)

/-! ## Theorem 1: partial correctness -/

theorem mem_listingsIn (a : α) (w : Window) :
    a ∈ listingsIn P loc w ↔ a ∈ P ∧ inWin (loc a) w := by
  simp [listingsIn, List.mem_filter]

theorem oracle_some {w : Window} {A : List α} (h : oracle P loc k w = some A) :
    A = listingsIn P loc w ∧ (listingsIn P loc w).length ≤ k := by
  unfold oracle at h
  split at h
  · cases h; exact ⟨rfl, by assumption⟩
  · cases h

theorem leafCount_singleton (p : Int × Int) (w : Window) : leafCount p [w] = ind p w := by
  simp only [leafCount, ind, List.countP_cons, List.countP_nil, decide_eq_true_eq]
  split <;> rfl
```

```
theorem leafCount_append (p : Int × Int) (l₁ l₂ : List Window) :
    leafCount p (l₁ ++ l₂) = leafCount p l₁ + leafCount p l₂ := by
  simp [leafCount, List.countP_append]

/-- A non-overflowing window is a correct leaf. -/
theorem leaf_correct {w : Window} {A : List α} (h : oracle P loc k w = some A) :
    Correct P loc k w A [w] := by
  obtain ⟨rfl, hk⟩ := oracle_some P loc k h
  refine ⟨fun a => mem_listingsIn P loc a w, ?_, fun p => leafCount_singleton p w⟩
  intro l hl
  simp only [List.mem_singleton] at hl
  subst hl; exact hk

/-- **Theorem 1.** Whenever `collect` returns an answer, the answer is correct. -/
theorem collect_correct (fuel : Nat) :
    ∀ (w : Window) (out : List α) (leaves : List Window),
      collect P loc k fuel w = some (out, leaves) → Correct P loc k w out leaves := by
  induction fuel with
  | zero =>
    intro w out leaves h
    unfold collect at h
    split at h
    · cases h; exact leaf_correct P loc k (by assumption)
    · cases h
  | succ fuel ih =>
    intro w out leaves h
    unfold collect at h
    split at h
    · cases h; exact leaf_correct P loc k (by assumption)
    · split at h
      · rename_i o1 l1 o2 l2 o3 l3 o4 l4 h1 h2 h3 h4
        cases h
        obtain ⟨m1, c1, v1⟩ := ih _ _ _ h1
        obtain ⟨m2, c2, v2⟩ := ih _ _ _ h2
        obtain ⟨m3, c3, v3⟩ := ih _ _ _ h3
        obtain ⟨m4, c4, v4⟩ := ih _ _ _ h4
        refine ⟨?_, ?_, ?_⟩
        · -- out = P_w: a listing is in some quadrant's output iff it is in w (Lemma 1).
          intro a
          rw [quadrants_cover]
          simp only [List.mem_append, m1, m2, m3, m4, and_or_left, or_assoc]
        · intro l hl
          simp only [List.mem_append] at hl
          rcases hl with ((hl | hl) | hl) | hl
          · exact c1 l hl
          · exact c2 l hl
          · exact c3 l hl
          · exact c4 l hl
        · intro p
          rw [leafCount_append, leafCount_append, leafCount_append, v1, v2, v3, v4]
          exact quadrants_partition p w
      · cases h

/-! ## Lemma 2, Lemma 3, Theorem 2: the depth budget `⌈log₂ W⌉` suffices -/

/-- `m ≤ k`: no single location holds more than `k` listings. -/
def CapOK : Prop := ∀ p : Int × Int, (P.filter (fun a => decide (loc a = p))).length ≤ k

/-- A filter by a stronger predicate is no longer than a filter by a weaker one. -/
theorem length_filter_le_of_imp (l : List α) (p q : α → Bool) (h : ∀ a, p a = true → q a = true) :
    (l.filter p).length ≤ (l.filter q).length := by
  induction l with
  | nil => simp
  | cons a l ih =>
    simp only [List.filter_cons]
    cases hp : p a with
    | true =>
      have hq := h a hp
      simp only [hq, ite_true, List.length_cons]
      omega
    | false =>
      simp only [Bool.false_eq_true, ite_false]
      split
      · simp only [List.length_cons]; omega
      · exact ih

/-- **Lemma 2.** A window of side at most 1 holds listings of a single
location, so under `m ≤ k` it is never split. -/
theorem unit_window_fits (hcap : CapOK P loc k) (w : Window)
    (hx : w.x1 - w.x0 ≤ 1) (hy : w.y1 - w.y0 ≤ 1) :
    (listingsIn P loc w).length ≤ k := by
  refine Nat.le_trans (length_filter_le_of_imp P _ _ ?_) (hcap (w.x0, w.y0))
```

```
  intro a ha
  simp only [decide_eq_true_eq, inWin, Prod.ext_iff] at ha ⊢
  omega

/-- **Lemma 3.** The quadrants of a window of side at most `2B` have side at most `B`. -/
theorem quadrants_side (w : Window) (B : Int)
    (hx : w.x1 - w.x0 ≤ 2 * B) (hy : w.y1 - w.y0 ≤ 2 * B) :
    ((q1 w).x1 - (q1 w).x0 ≤ B ∧ (q1 w).y1 - (q1 w).y0 ≤ B) ∧
    ((q2 w).x1 - (q2 w).x0 ≤ B ∧ (q2 w).y1 - (q2 w).y0 ≤ B) ∧
    ((q3 w).x1 - (q3 w).x0 ≤ B ∧ (q3 w).y1 - (q3 w).y0 ≤ B) ∧
    ((q4 w).x1 - (q4 w).x0 ≤ B ∧ (q4 w).y1 - (q4 w).y0 ≤ B) := by
  simp only [q1, q2, q3, q4]
  omega

/-- **Theorem 2.** Under `m ≤ k`, on a window of side at most `2^D`, `collect`
with depth budget `D` returns an answer. -/
theorem collect_terminates (hcap : CapOK P loc k) (D : Nat) :
    ∀ w : Window, w.x1 - w.x0 ≤ (2 : Int) ^ D → w.y1 - w.y0 ≤ (2 : Int) ^ D →
      ∃ r, collect P loc k D w = some r := by
  induction D with
  | zero =>
    intro w hx hy
    simp only [Int.pow_zero] at hx hy
    have hfits := unit_window_fits P loc k hcap w hx hy
    refine ⟨(listingsIn P loc w, [w]), ?_⟩
    simp [collect, oracle, hfits]
  | succ D ih =>
    intro w hx hy
    rw [Int.pow_succ, Int.mul_comm] at hx hy
    obtain ⟨⟨a1, b1⟩, ⟨a2, b2⟩, ⟨a3, b3⟩, ⟨a4, b4⟩⟩ := quadrants_side w _ hx hy
    unfold collect
    split
    · exact ⟨_, rfl⟩
    · obtain ⟨⟨o1, l1⟩, h1⟩ := ih (q1 w) a1 b1
      obtain ⟨⟨o2, l2⟩, h2⟩ := ih (q2 w) a2 b2
      obtain ⟨⟨o3, l3⟩, h3⟩ := ih (q3 w) a3 b3
      obtain ⟨⟨o4, l4⟩, h4⟩ := ih (q4 w) a4 b4
      rw [h1, h2, h3, h4]
      exact ⟨_, rfl⟩

/-- **Corollary 1 (total correctness).** Under `m ≤ k`, on a window of side at
most `2^D`, `collect` with depth budget `D` returns a correct answer. -/
theorem collect_total (hcap : CapOK P loc k) (D : Nat) (w : Window)
    (hx : w.x1 - w.x0 ≤ (2 : Int) ^ D) (hy : w.y1 - w.y0 ≤ (2 : Int) ^ D) :
    ∃ out leaves, collect P loc k D w = some (out, leaves) ∧ Correct P loc k w out leaves := by
  obtain ⟨⟨out, leaves⟩, h⟩ := collect_terminates P loc k hcap D w hx hy
  exact ⟨out, leaves, h, collect_correct P loc k D w out leaves h⟩

end Quadtree
```

APPENDIX C
LLM USAGE: DIVIDE-AND-CONQUER PART

Tool. Claude Code (Anthropic's command-line coding agent), model Claude Opus 5.5 (`claude-opus-5-5`), October 2026. The agent read the assignment, read the scraper repository [1], searched the web for the IEEE template, wrote and ran the code, and drafted this section. All proofs, numbers, and citations were checked by the authors.

Prompts (verbatim).

1) "DO you understand the project I need to do also this is a group project I am doing the divide an conquer part"
2) "My Idea for the problem is a problem I have solved before with divide and conquer https://github.com/IlanDanial/airbnb_data I split the map over and over, each time checking the amount of properties in the split and if it was under I took those properties and added it to the list since you can only see so many properties at a time"
3) "yes start the implementation, we're using IEEE search online for proper template"
4) "yes write the LaTeX section"
5) "This is really ugly" followed by "My bigger issue is that the explanations are very longwinded for no reason and the formatting of the page is ugly its hard to read as well"
6) "I have an issue with some of this read the paper be critical how do you think of the formatting", then "fix all of them at once" and "also do we need the full code in the appendix or can we reference a repository"
7) "would it be worth it to look at the code and refactor it and the comments in the code", then "go"
8) "Isnt there a code languge that is built to prove proofs can we use that to prove correctness", then "I want to go for it also I am sure there are online proofs of this exact same thing in the same language so it should take less than an hour"
9) "so your telling me that there was no problem with proving the whole thing end to end"
10) "actually search online for the hard part dont just tell me it does not exist"

11) “do both, go for the full proof, or is there a solution we can do that would be easier to prove and solve the problem all the same”
12) “why would anything in the paper not exactly match the proof we did with elon”, then “we need to have the paper be matching hte lean”
13) “can we get rid of common AI’sms like mdashes list a couple more ai isms”
14) Two screenshots of the PDF with “why is this structure so odd”, “things are so packed together that it is unreadable, also the – looks like m dashes”, a request that only the partner’s remaining work be marked as to do, and the author’s affiliation details.
15) “whats left for me to do”, “according to the instructions it seems like it is asking me to list everything in the appendix”, and the decision to export the transcript into the repository rather than typeset it here.

Full transcript. The complete session, exported with Claude Code’s `/export` command, is in the repository at `divide_and_conquer/appendix/transcript.md` (commit `cba125c`), with every prompt and response in order. Intermediate versions of the code, figures and this report are in the repository’s commit history.

Results. (1) A summary of the assignment and four candidate divide-and-conquer problems. (2) An analysis of the scraper’s `subdivide` recursion as an adaptive quadtree, identifying the termination issue for co-located listings and the double-counting issue on window borders. (3) `quadtree.py`, `test_quadtree.py`, `experiments.py`, and the figures; an intermediate run stopped when the minimum-width guard was reached, after which the crowding experiment was limited to $\delta \geq 2^{-28}$, and garbage collection was disabled during timing. (4) A first draft of Section III and these appendices, with five separate figures in matplotlib’s default style. (5) The draft was shortened by about a third, the five figures were restyled to match the paper’s typeface and merged into three, and the page layout was fixed. (6) A list of nine formatting defects (an equation label pushed to its own line, a wrapped pseudocode line, mixed paragraph styles, inconsistent equation citations, a legend mismatch), all then fixed; the unit tests were removed from this appendix in favor of the repository link. (7) The plotting code was split out of `experiments.py` into `plots.py`, and docstrings and comments were added. (8) An assessment of proof assistants (Lean 4, Coq, Isabelle, Dafny) and of which parts of the proof are feasible to formalize; then a first Lean formalization of Lemma 1 and Theorem 1 only, with COLLECT given a fuel bound, which compiled on the first build. (9) A clarification that this was partial correctness, not an end-to-end proof, with three further gaps listed (integer midpoints, the oracle not being a black box, no duplicate-freeness proof). (10) A web search that found the Isabelle and Coq formalizations cited in Section III-D, none covering separation-based termination. The assistant’s earlier claim that no related formalization existed was withdrawn. (11) The observation that finite-precision coordinates make the problem a grid problem, on which Theorem 2 is provable by induction on the depth; `collect_terminates` and `collect_total` were then added, needing one round of fixes to core-library lemma names before compiling. (12) A table of the remaining differences between the paper’s model (real coordinates, exact midpoints, leaves asserted disjoint) and the Lean model; the paper’s abstraction and Section III-D were then rewritten on the integer grid to match the Lean file statement for statement, and leaf disjointness was added to the Lean proof (`quadrants_partition` in counting form). (13) A scan of the report for colon-spliced sentences, filler adverbs and similar tics, which were removed; no em dashes were present. (14) Theorem spacing via `amsthm`, numeric ranges rewritten in words, the front matter and introduction and conclusion for this half written, and the partner’s remaining items marked in red. (15) The advice that the full transcript must be included rather than attached, then the pointer above once the transcript was exported.